

Parallel Foreach Loop in C#

In this article, I will discuss the **Parallel Foreach Loop in C#** with Examples. As we already discussed in our previous article, the **Task Parallel Library (TPL)** provides two methods (i.e., **Parallel For** and **Parallel Foreach**), which are conceptually the “for” and “for each” loops, except that they use multiple threads to execute multiple iterations at the same time on a machine with multiple cores. In our previous article, we already discussed the [Parallel for Method in C#](#) with examples.

Parallel For Each Loop in C#

The Parallel For Each Loop in C# is a part of the Task Parallel Library (TPL) and is used for parallel processing. It allows us to execute iterations of a loop concurrently, utilizing multiple threads and cores of the processor. This can significantly improve performance for CPU-bound operations or tasks that can be executed concurrently.

The Parallel Foreach Method in C# provides a parallel version of the standard, sequential Foreach loop. In a standard Foreach loop, each iteration processes a single item from the collection and will only process all the items one by one. However, the Parallel Foreach method executes multiple iterations simultaneously on different processors or processor cores. This may open the possibility of synchronization problems. So, the loop is ideally suited to processes where each iteration is independent.

A Sequential For Each Loop Syntax in C#:

```
List<int> integerList = Enumerable.Range(1, 10).ToList();
foreach (int i in integerList)
{
    //...
};
```

A Parallel Foreach Loop Syntax in C#:

```
List<int> integerList = Enumerable.Range(1, 10).ToList();
Parallel.ForEach(integerList, i =>
{
    //...
});
```

The parallel version of the loop uses the static Foreach method of the Parallel class. There are many overloaded versions available for this method. This is the simplest overloaded version that accepts two arguments.

- The first parameter is the collection of objects that will be enumerated. This can be any collection that implements `IEnumerable<T>`.
- The second parameter accepts an Action delegate, usually expressed as a lambda expression that determines the action to take for each item in the collection.

Parallel Foreach Loop Example in C#

Let us understand the Parallel Foreach Method in C# with an example. First, we will write the example using the standard sequential Foreach loop and will see how much time it will take to complete the execution. Then, we will write the same example using the Parallel Foreach Loop method and see how long it will take to complete the execution of the same example.

In the below example, we create a sequential Foreach Loop that performs a long-running task once for each item in the collection. The code below loops through a list of ten integers generated using the **Enumerable.Range** method. In each iteration, the **DoSomeIndependentTimeConsumingTask** method is called. The **DoSomeIndependentTimeConsumingTask** method performs a calculation that is included to generate a long enough pause to see the performance improvement of the parallel version.

Example using Standard Foreach Loop in C#:

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main()
        {
            Stopwatch stopwatch = new Stopwatch();
            Console.WriteLine("Standard Foreach Loop Started");
            stopwatch.Start();
            List<int> integerList = Enumerable.Range(1, 10).ToList();
            foreach (int i in integerList)
            {
                long total = DoSomeIndependentTimeConsumingTask();
                Console.WriteLine("{0} - {1}", i, total);
            };
            Console.WriteLine("Standard Foreach Loop Ended");
            stopwatch.Stop();

            Console.WriteLine($"Time Taken by Standard Foreach Loop in Miliseconds: {stopwatch.ElapsedMilliseconds}");
            Console.ReadLine();
        }
        static long DoSomeIndependentTimeConsumingTask()
        {
            //Do Some Time Consuming Task here
            long total = 0;
            for (int i = 1; i < 100000000; i++)
            {
                total += i;
            }
            return total;
        }
    }
}
```

```

    }
}
}

```

Now, run the application and observe the output.

```

Standard Foreach Loop Started
1 - 4999999950000000
2 - 4999999950000000
3 - 4999999950000000
4 - 4999999950000000
5 - 4999999950000000
6 - 4999999950000000
7 - 4999999950000000
8 - 4999999950000000
9 - 4999999950000000
10 - 4999999950000000
Standard Foreach Loop Ended
Time Taken by Standard Foreach Loop in Milliseconds 2305

```

As you can see from the above output, the standard Foreach Loop statement took approximately 2305 milliseconds to complete the execution. Let's rewrite the same example using the C# Parallel ForEach method.

Example using Parallel Foreach Loop in C#:

Let's rewrite the previous example using the Parallel ForEach Loop and see the output.

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main()
        {
            Stopwatch stopwatch = new Stopwatch();
            Console.WriteLine("Parallel Foreach Loop Started");
            stopwatch.Start();
            List<int> integerList = Enumerable.Range(1, 10).ToList();
            Parallel.ForEach(integerList, i =>
            {
                long total = DoSomeIndependentTimeConsumingTask();
                Console.WriteLine("{0} - {1}", i, total);
            });
            Console.WriteLine("Parallel Foreach Loop Ended");
        }
    }
}

```

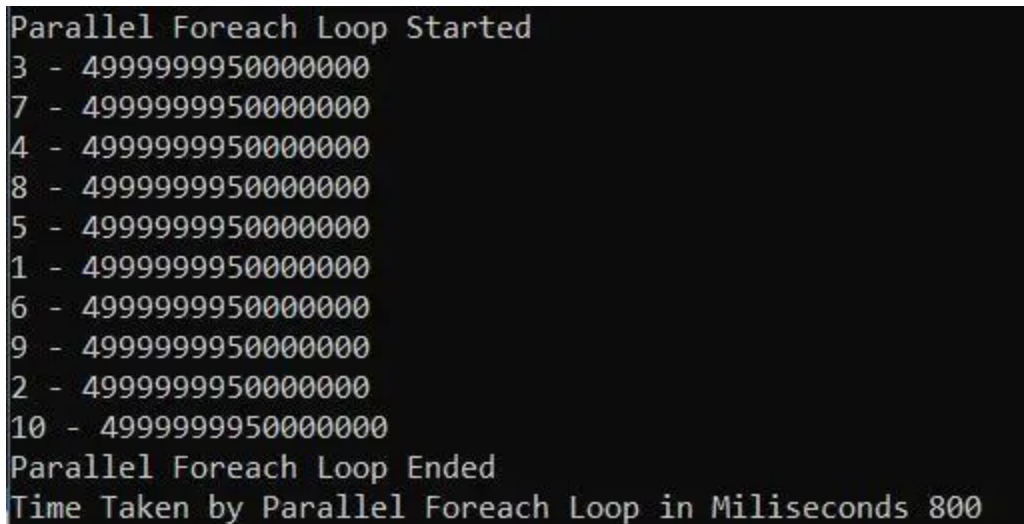
```

        stopwatch.Stop();

        Console.WriteLine($"Time Taken by Parallel Foreach Loop in Miliseconds {stopwatch.ElapsedMilliseconds}");
        Console.ReadLine();
    }
    static long DoSomeIndependentTimeConsumingTask()
    {
        //Do Some Time Consuming Task here
        long total = 0;
        for (int i = 1; i < 100000000; i++)
        {
            total += i;
        }
        return total;
    }
}

```

Now, run the above code and see the output as shown below. The time may vary on your machine.



```

Parallel Foreach Loop Started
3 - 4999999950000000
7 - 4999999950000000
4 - 4999999950000000
8 - 4999999950000000
5 - 4999999950000000
1 - 4999999950000000
6 - 4999999950000000
9 - 4999999950000000
2 - 4999999950000000
10 - 4999999950000000
Parallel Foreach Loop Ended
Time Taken by Parallel Foreach Loop in Miliseconds 800

```

As you can see in the above output, the Parallel ForEach method took 800 milliseconds to complete the execution compared to 2305 milliseconds with the standard Foreah loop in C#.

Using Degree of Parallelism in C# with Parallel Foreach Loop:

Using the Degree of Parallelism in C#, we can specify the maximum number of threads to execute the parallel for each loop. The syntax for using the Degree of Parallelism in C# is given below.

```

var options = new ParallelOptions()
{
    MaxDegreeOfParallelism = 2
};
List<int> integerList = Enumerable.Range(1, 10).ToList();
Parallel.ForEach(integerList, options, i =>
{
    long total = DoSomeIndependentTimeconsumingTask();
    Console.WriteLine("{0} - {1}", i, total);
});

```

The MaxDegreeOfParallelism property of the ParallelOptions class specifies the number of concurrent operations the Parallel method runs. Then, we need to pass this instance to the Parallel ForEach method. A positive property value limits the number of concurrent operations to the set value. If it is -1, there is no limit on the number of concurrently running operations.

By default, For and ForEach will utilize however many threads the underlying scheduler provides, so changing MaxDegreeOfParallelism from the default only limits how many concurrent tasks will be used.

Example to Understand Degree of Parallelism in C#

Let us see an example for a better understanding. In the below example, we are executing the Parallel Foreach method without using the Degree of Parallelism. That means we are not limiting the number of threads to execute the Parallel Foreach method.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main()
        {
            List<int> integerList = Enumerable.Range(0, 10).ToList();
            Parallel.ForEach(integerList, i =>
            {
                Console.WriteLine(@"value of i = {0}, thread = {1}", i,
Thread.CurrentThread.ManagedThreadId);
            });
            Console.ReadLine();
        }
    }
}

```

Output:

```
value of i = 0, thread = 1
value of i = 3, thread = 1
value of i = 4, thread = 5
value of i = 6, thread = 5
value of i = 7, thread = 5
value of i = 8, thread = 5
value of i = 9, thread = 5
value of i = 5, thread = 1
value of i = 2, thread = 4
value of i = 1, thread = 3
```

Now run the above code multiple times, and you will definitely get different output. You will also observe that the number of threads created is not in our control. In my case, 4 threads run the parallel for each loop. In your case, the number of threads might vary. Now, let us see how to restrict the number of threads to be created.

How to Control the Degree of Concurrency, i.e., How to Restrict the Number of Threads Created?

We can restrict the number of concurrent threads created during the execution of a parallel loop by using the `MaxDegreeOfParallelism` property of the `ParallelOptions` class. By assigning some integer value to `MaxDegreeOfParallelism`, we can restrict the degree of this concurrency and the number of processor cores used by our loops. The default value of this property is -1, which means there is no restriction on concurrently running operations.

Example using Degree of Parallelism in C# to Restrict the number of Threads

In the below example, we have set `MaxDegreeOfParallelism` to 2, which means a maximum of 2 threads will execute our parallel for each loop.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main()
        {
            List<int> integerList = Enumerable.Range(0, 10).ToList();
            var options = new ParallelOptions() { MaxDegreeOfParallelism = 2 };
            Parallel.ForEach(integerList, options, i =>
            {
                Console.WriteLine(@"value of i = {0}, thread = {1}", i,
Thread.CurrentThread.ManagedThreadId);
            });
            Console.ReadLine();
        }
    }
}
```

```
}  
}  
}
```

Now run the application and see the output as shown below. Whatever number of times we execute the above code, the number of threads will never exceed 2.

```
value of i = 5, thread = 3  
value of i = 6, thread = 3  
value of i = 7, thread = 3  
value of i = 8, thread = 3  
value of i = 9, thread = 3  
value of i = 1, thread = 3  
value of i = 2, thread = 3  
value of i = 3, thread = 3  
value of i = 4, thread = 3  
value of i = 0, thread = 1
```

Speed Benefits of Parallelism in C#:

We have already understood that increasing speed is the most important reason for using parallelism. We have seen several examples where we compare the sequential and parallel execution of an algorithm, and we have always seen a decrease in the execution time of the program by using parallelism. In other words, we have consistently obtained better results when using parallelism.

However, as we know, nothing is free in this life, and parallelism is not the exception. We will not always obtain better results when introducing parallelism in our applications. This is because there is a cost to preparing the use of multithreading. That is why it is always advisable to take measurements to see whether the use of parallelism exceeds the cost.

Is it Worth using Parallelism in C#?

We can make an analogy. If you are a teacher who has to correct one exam, let's say that it takes you four minutes to correct a test. Let's also assume that finding two helpers takes 45 minutes, and each helper takes four minutes to correct the exam.

Correct an exam = 4 minutes

Find 2 assistants or helpers = 45 minutes

Is it worth hiring a helper for this task? If you spend 45 minutes finding two helpers or two assistants and then give the task to one of them to correct that, it will take him 4 minutes to correct that. The total time of the task, adding the 45 minutes of looking for assistance and four minutes of correcting this time, adds up to 49 minutes, which is more than the four minutes it would have taken to correct the exam by yourself.

Correct 1 exam

You alone: $4 \times 1 = 4$ minutes

In group: $45 + 4 = 49$ minutes

As you can see, working with assistants took longer than working alone. The cost of this is the small number of tests to correct. Suppose that instead of one exam, there were 150 exams. So, on your own or alone, it will take 600 minutes to correct them. But together with your assistants, the time will be only 245 minutes.

Correct 150 exams

You alone: $4 \times 150 = 600$ minutes

In group: $45 + 4 \times 50 = 245$ minutes

As you can see in the second case, having the systems paid off, even considering the 45 minutes it took to hire those assistants.

Something similar happens with Parallelism. Sometimes, the work is so small, so little, that it is faster to use Sequential Programming and not Parallel Programming. The important thing is to take measurements before and after introducing parallelism to make sure that parallelism really pays off.

Example For a Better Understanding:

Please have a look at the below example. In the example below, the same task will be performed using C# Standard For Loop and Parallel Foreach Loop. But here, the task is not expensive or time-consuming. It is just a simple task. Now, if you run the code, you will observe that the Parallel version of the foreach loop takes more time than the standard foreach loop. This is because the parallel foreach creates multiple threads, which will take some time, which is not in the case of a standard foreach loop, as a single thread will execute the tasks.

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main()
        {
            Stopwatch stopwatch = new Stopwatch();
            Console.WriteLine("Standard Foreach Loop Started");
            stopwatch.Start();
            List<int> integerList = Enumerable.Range(1, 10).ToList();
            foreach (int i in integerList)
            {
```

```

        DoSomeIndependentTask(i);
    };

    stopwatch.Stop();
    Console.WriteLine("Standard Foreach Loop Ended");
    Console.WriteLine($"Time Taken by Standard Foreach Loop in Miliseconds: {stopwatch.ElapsedMilliseconds}");
    Console.WriteLine("\nParallel Foreach Loop Started");
    stopwatch.Restart();

    Parallel.ForEach(integerList, i =>
    {
        DoSomeIndependentTask(i);
    });

    stopwatch.Stop();
    Console.WriteLine("Parallel Foreach Loop Ended");
    Console.WriteLine($"Time Taken by Parallel Foreach Loop in Miliseconds: {stopwatch.ElapsedMilliseconds}");

    Console.ReadLine();
}
static void DoSomeIndependentTask(int i)
{
    Console.WriteLine($"Number: {i}");
}
}
}

```

Output:

```
Standard Foreach Loop Started
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Number: 6
Number: 7
Number: 8
Number: 9
Number: 10
Standard Foreach Loop Ended
Time Taken by Standard Foreach Loop in Milliseconds 1

Parallel Foreach Loop Started
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Number: 6
Number: 7
Number: 8
Number: 9
Number: 10
Parallel Foreach Loop Ended
Time Taken by Parallel Foreach Loop in Milliseconds 23
```

As you can see in the above image, in my machine, the standard for each loop took 1 second compared with 23 seconds with the Parallel for each loop. So, this proves that the Parallel Foreach loop does not always give you better performance. So, you have to take measurements before and after introducing parallelism to ensure that parallelism gives you better performance.

When to use Parallel Foreach Loop in C#?

Using the Parallel ForEach loop in C# can significantly improve performance for data processing tasks by concurrently utilizing multiple processor cores. However, it's not always appropriate for every situation. Here are some guidelines to help you decide when to use Parallel.ForEach:

- **Independent Operations:** Ideal for scenarios where you have a collection of items to process, and each item can be processed independently without affecting the others.
- **CPU-Bound Tasks:** Best suited for CPU-intensive tasks that can benefit from parallel execution. For example, complex calculations on elements of a large array.
- **Performance Optimization:** When a performance bottleneck in a loop processes many items or performs time-consuming operations, profiling indicates that parallelization could help.
- **Large Collections:** Useful when dealing with large collections where the performance gains from concurrent processing justify parallelization overhead.
- **Non-Blocking Operations:** When the operations within the loop do not involve blocking calls (like I/O operations). Blocking calls in a parallel loop can negate the benefits of parallelism and lead to thread

pool starvation.

Considerations and Best Practices

- **Avoid Shared State:** Ensure that the body of the loop does not depend on a shared state or has appropriate synchronization mechanisms to handle shared state safely.
- **Handle Exceptions Carefully:** Exceptions thrown in a Parallel ForEach loop need to be handled within the loop. AggregateException is often used to catch multiple exceptions that might occur in parallel tasks.
- **Beware of I/O Bound Operations:** I/O-bound operations typically do not benefit much from parallelization and can lead to resource contention.
- **Balance Between Parallelism and Overhead:** Over-parallelizing (especially with many lightweight tasks) can lead to excessive context switching and thread management overhead, reducing overall performance.

Advantages and Disadvantages of using Parallel Foreach Loop in C#

Parallel ForEach in C# can offer several advantages and disadvantages, depending on the specific use case and its implementation. Here's an overview of the advantages and disadvantages:

Advantages of Using Parallel ForEach Loop in C#:

- **Improved Performance:** The most significant advantage is the potential for a dramatic increase in performance, especially for CPU-bound tasks. Parallel ForEach can significantly reduce the time it takes to process large collections by parallelizing work across multiple cores.
- **Simplified Parallelism:** Parallel ForEach abstracts much of the complexity of manual thread management and synchronization, making it easier to implement parallel processing.
- **Scalability:** It automatically scales the degree of parallelism, meaning it can adjust the number of threads it uses based on the capabilities of the host system.
- **Flexibility and Customization:** Offers options to configure the behavior of the parallel loop, such as setting the maximum degree of parallelism, cancellation support, and exception handling.
- **Better Resource Utilization:** Efficient use of available processors, leading to better overall system resource utilization.

Disadvantages of Using Parallel ForEach Loop in C#:

- **Overhead:** Parallel processing adds overhead due to thread creation and management, context switching, and potential lock contention. For small collections or fast operations, the overhead might outweigh the benefits.
- **Complex Debugging and Testing:** Debugging parallel code can be more complex than sequential code. Race conditions, deadlocks, and other concurrency-related bugs can be challenging to identify and fix.
- **Shared State Management:** Handling shared state in parallel loops requires careful synchronization, which can introduce complexity and the risk of concurrency-related issues like deadlocks or race conditions.
- **Non-Deterministic Behavior:** The order of operation execution is not deterministic in a parallel loop, which can be problematic for certain algorithms that require ordered execution.
- **Limited Benefit for I/O-Bound Tasks:** I/O-bound operations often do not benefit much from parallel execution and might be better handled using asynchronous programming patterns.

In the next article, I will discuss the [Parallel Invoke Method in C#](#) with Examples. In this article, I try to explain the Parallel ForEach in C# with Examples. I hope you understand the need and use of the Parallel Foreach Method in C#.